

Trabajo Práctico N.º 4: Programación con FreeRTOS

Braida Agustín (404621) - Ernst Pedro (400624) - Polizzi Tomás (400793) - Rao Valentino (402308)

*Técnicas Digitales II 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional*

Resumen—En el presente trabajo práctico se aborda el diseño, configuración e implementación de aplicaciones embebidas multitarea en tiempo real sobre el microcontrolador STM32F103C8T6 (núcleo ARM Cortex-M3 @ 72 MHz) utilizando el sistema operativo FreeRTOS v10.x en conjunto con las librerías CMSIS. En una primera instancia, se analiza la infraestructura de software modular desarrollada como base en el Trabajo Práctico N.º 3, detallando la interacción entre el gestor de memoria dinámica heap_4.c, la capa portable para ARMv7-M y los manejadores de excepciones del núcleo (SVC, PendSV y SysTick). Posteriormente, en el Ejercicio N.º 1, se implementa un esquema de concurrencia preemptivo por prioridades que integra un parpadeo de LED a 5 Hz, telemetría periódica por USART1 a 115200 bps y un monitor de carga de CPU mediante la función vApplicationIdleHook(). Finalmente, en el Ejercicio N.º 2, se resuelve el problema clásico de sincronización y comunicación entre productor y consumidor, coordinando la digitalización periódica de un canal analógico (ADC1 en PA4) mediante un semáforo binario en la rutina de interrupción (ADC1_2_IRQHandler) y la transferencia segura de muestras por copia de valor a través de una cola (Queue) hacia una tarea consumidora encargada de transmitir los datos procesados por la interfaz serie.

Index Terms—FreeRTOS, ARM Cortex-M3, STM32F103, CMSIS, Planificación Preemptiva, Semáforos Binarios, Colas de Mensajes, Productor-Consumidor, Tiempo Real.

I. INTRODUCCIÓN

EL desarrollo tradicional de sistemas embebidos bajo la arquitectura *Bare-Metal* estructura habitualmente el flujo de ejecución principal mediante un lazo infinito (*Super-Loop*) complementado con rutinas de servicio de interrupción (ISRs). Si bien este enfoque resulta eficiente en términos de memoria en aplicaciones elementales, presenta severas limitaciones de escalabilidad cuando el sistema debe coordinar múltiples tareas concurrentes con restricciones temporales críticas. El uso de demoras por espera activa (*polling*) o funciones de retardo bloqueantes introduce variabilidad temporal acumulativa (*jitter*) y monopoliza la CPU de manera improductiva, degradando el determinismo del sistema.

Para superar estas limitaciones, se introduce el paradigma de los **Sistemas Operativos en Tiempo Real** (*Real-Time Operating Systems* – RTOS). Un RTOS descompone la aplicación en un conjunto de hilos o **Tareas** (*Tasks*) independientes, dotadas de su propia pila de ejecución (*Stack*) y nivel de prioridad. El núcleo del sistema operativo incorpora un **Planificador** (*Scheduler*) que asigna el uso de la CPU en base a eventos y prioridades, garantizando que ante el requerimiento de una tarea de alta prioridad, el procesador responda de forma determinista y predecible.

En este trabajo práctico, se emplea el kernel **FreeRTOS** integrado sobre la plataforma de hardware **BluePill** (STM32F103C8T6), explorando los conceptos de multitarea cooperativa y preemptiva, monitoreo de actividad del procesador mediante tareas inactivas (*Idle Hook*), y patrones formales de sincronización y paso de mensajes entre interrupciones y tareas mediante semáforos binarios y colas (*Queues*).

II. FUNDAMENTOS DE LA ESTRUCTURA DE FREERTOS (BASE DEL TP3)

Para posibilitar la ejecución del sistema operativo sobre el hardware del STM32F103, en el Trabajo Práctico N.º 3 se construyó una estructura de proyecto modular e independiente que vincula las capas estándar de ARM (CMSIS), el código del fabricante (CMSIS-Device-F1) y el núcleo de **FreeRTOS**. La comprensión de esta infraestructura es fundamental para el desarrollo del presente informe.

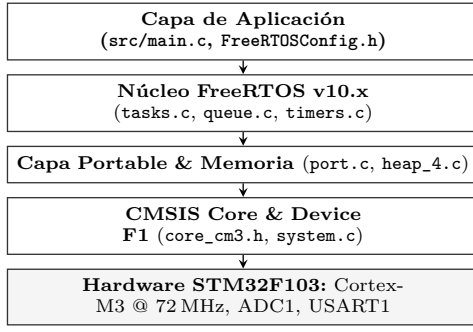


Figura 1. Diagrama de capas de software y abstracción implementadas.

II-A. Integración de CMSIS y Configuración de Reloj a 72 MHz

El estándar CMSIS proporciona las definiciones de registros del núcleo Cortex-M3 (`core_cm3.h`) y los periféricos específicos del STM32F103 (`stm32f103xb.h`). En el archivo `system_stm32f1xx.c`, la función `SystemInit()` se encarga de configurar el árbol de reloj (*Clock Tree*) inmediatamente tras el reinicio:

1. Habilita el oscilador de cristal externo HSE de 8 MHz (`RCC_CR_HSEON`) y aguarda su estabilización.
2. Configura la memoria Flash con 2 estados de espera (`FLASH_ACR_LATENCY_2`) y habilita el buffer de prebúsqueda (`PRFTBE`), requisito físico obligatorio para operar a frecuencias superiores a 48 MHz.
3. Ajusta los divisores de bus: AHB en división por 1 (72 MHz), APB1 en división por 2 (36 MHz máx.) y APB2 en división por 1 (72 MHz).
4. Configura el multiplicador PLL en $\times 9$ ($HSE \times 9 = 72 \text{ MHz}$) y conmuta la señal de reloj del sistema (`SYSCCLK`) hacia la salida del PLL.

II-B. Capa Portable y Enlace de Excepciones del Núcleo

FreeRTOS requiere interactuar con el hardware del procesador para realizar los cambios de contexto y la temporización. En la arquitectura ARMv7-M, esto se implementa en `FreeRTOS/portable/GCC/ARM_CM3/port.c` mediante tres excepciones:

- **SVC** (`vPortSVCHandler`): Excepción disparada por software mediante la instrucción `SVC 0` para iniciar la primera tarea del sistema, pasando del modo privilegiado al modo de usuario con el Process Stack Pointer (PSP).
- **SysTick** (`xPortSysTickHandler`): Temporizador periódico de 24 bits configurado a 1000 Hz (1 ms por tick). Incrementa el contador global de tiempo (`xTickCount`) y evalúa si corresponde desbloquear tareas dormidas.
- **PendSV** (`xPortPendSVHandler`): Excepción de servicio postergable configurada con la menor prioridad posible del sistema. Se encarga de salvar en el stack de la tarea saliente los registros R4-R11, conmutar el puntero de pila PSP hacia la tarea entrante y restaurar su contexto, asegurando que el cambio de contexto

ocurra únicamente cuando no haya interrupciones de hardware anidadas activas.

Para vincular estos nombres con la tabla de vectores definida en `startup_stm32f103xb.s`, en `FreeRTOSConfig.h` se definen las macros de mapeo:

```
#define vPortSVCHandler    SVC_Handler
#define xPortPendSVHandler PendSV_Handler
#define xPortSysTickHandler SysTick_Handler
```

II-C. Gestión de Memoria Dinámica con `heap_4.c`

Para la asignación del bloque de control de tareas (TCB) y las pilas de ejecución, se utilizó el esquema `heap_4.c`. A diferencia de implementaciones más simples como `heap_1.c` (que no permite liberar memoria) o `heap_2.c` (que no fusiona bloques adyacentes), `heap_4.c` implementa un algoritmo de primer ajuste (*first-fit*) con **coalescencia de memoria**, combinando automáticamente bloques de memoria libres contiguos para evitar la fragmentación del Heap. Se reservó un espacio de 5 KB (`configTOTAL_HEAP_SIZE = 5120`) dentro de la memoria SRAM disponible.

II-D. Archivo de Configuración: `FreeRTOSConfig.h`

El comportamiento del kernel se parametriza mediante macros en `FreeRTOSConfig.h`. En la Tabla I se resumen los valores adoptados para la plataforma STM32F103.

Tabla I. Parámetros fundamentales en `FreeRTOSConfig.h`

Macro	Valor	Descripción Técnica
<code>configCPU_CLOCK_HZ</code>	72000000UL	Reloj CPU Cortex-M3.
<code>configTICK_RATE_HZ</code>	1000	Tick 1 ms del scheduler.
<code>configUSE_PREEMPTION</code>	1	Planificación preemptiva.
<code>configMAX_PRIORITIES</code>	5	Prioridades (0 a 4).
<code>configTOTAL_HEAP_SIZE</code>	5120	5 KB para el Heap.
<code>configKERNEL_INTERRUPT...</code>	255	Prioridad mínima (SysTick).
<code>configMAX_SYSCALL...</code>	191	Umbral para API segura.

III. EJERCICIO PRÁCTICO N.º 1: TAREAS, PLANIFICACIÓN Y MONITOR DE SISTEMA

III-A. Objetivo y Descripción de Tareas

El objetivo del primer ejercicio consistió en implementar un esquema de multitarea concurrente demostrando la cooperación entre tareas de diferente jerarquía de prioridad y la medición de la disponibilidad de CPU mediante la tarea inactiva (*Idle Task*).

Se crearon tres tareas concurrentes:

- **Tarea A** (`vTaskBlinky`) – **Prioridad 2 (Alta)**: Controla el LED integrado en la placa BluePill conectado al pin PC13 (activo en nivel bajo). Oscila a una frecuencia de **5 Hz** (período total de 200 ms: 100 ms en nivel bajo y 100 ms en nivel alto).
- **Tarea B** (`vTaskTelemetry`) – **Prioridad 1 (Baja)**: Transmite periódicamente cada **2 segundos** una trama de texto fija a través del periférico USART1

(PA9 TX a 115200bps, 8N1) informando el estado del sistema y la cuenta de actividad del procesador.

- **Tarea C (vApplicationIdleHook) – Prioridad 0 (Mínima):** Función de enganche ejecutada en cada ciclo de la tarea Idle cuando no existen tareas de usuario listas para correr, incrementando una variable entera volátil de 32 bits (ulIdleCycleCount).

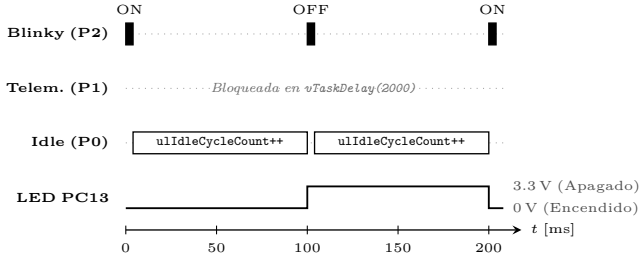


Figura 2. Cronograma temporal de tareas concurrentes y señal del LED a 5 Hz (Ejercicio 1).

III-B. Implementación y Código Fuente

En el Listado 1 se presenta la implementación de las tareas y la configuración del hardware en `act1/src/main.c`.

Listing 1. Fragmento principal del Ejercicio 1 (`act1/src/main.c`)

```
volatile uint32_t ulIdleCycleCount = 0;

// Tarea A: Blinky en PC13 a 5 Hz (Periodo 200 ms)
static void vTaskBlinky(void *pvParameters) {
    (void)pvParameters;
    for (;;) {
        GPIOC->BSRR = GPIO_BSRR_BR13; // LED ON
        vTaskDelay(pdMS_TO_TICKS(100));
        GPIOC->BSRR = GPIO_BSRR_BS13; // LED OFF
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// Tarea B: Telemetria UART cada 2 segundos
static void vTaskTelemetry(void *pvParameters) {
    (void)pvParameters;
    char buffer[64];
    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(2000));
        snprintf(buffer, sizeof(buffer),
            "Sistema_Operativo_Ejecutando_Idle: %lu\r\n",
            (unsigned long)ulIdleCycleCount);
        uart_send_str(buffer);
    }
}

// Tarea C: Hook de la Tarea Idle
void vApplicationIdleHook(void) {
    ulIdleCycleCount++;
}
```

III-C. Análisis Conceptual: ¿Por qué se diseñó de esta forma?

La arquitectura de este ejercicio se basa en los siguientes fundamentos:

1. **Retardo no bloqueante con vTaskDelay():** Al invocar `vTaskDelay(pdMS_TO_TICKS(100))`, la Tarea A no retiene la CPU en un bucle ocupado; en su lugar, el kernel cambia su estado de *Running* a *Blocked* y la remueve de la lista de tareas listas (*Ready List*). Esto permite que el planificador ceda inmediatamente el

control a las tareas de menor prioridad (Tarea B e Idle Task).

2. **Preempción por Prioridad:** Cuando transcurren los 100 ms, la interrupción de SysTick transfiere a la Tarea A nuevamente al estado *Ready*. Dado que posee Prioridad 2 (superior a la Prioridad 1 de la Tarea B y Prioridad 0 de Idle), el planificador desaloja de forma inmediata a la tarea en ejecución y conmuta el procesador hacia la Tarea A, garantizando la exactitud del parpadeo a 5 Hz.
3. **Medición de Carga de CPU con Idle Hook:** Debido a que las Tareas A y B pasan la gran mayoría del tiempo bloqueadas esperando eventos temporales (la Tarea A ejecuta apenas decenas de instrucciones cada 100 ms y la Tarea B formatea texto cada 2 s), la CPU permanece en estado ocioso más del 99,5 % del tiempo. Durante esos periodos, la tarea Idle ejecuta de forma continua `vApplicationIdleHook()`, incrementando `ulIdleCycleCount` a un ritmo monotónicamente creciente ($\sim 1,4 \times 10^6$ ciclos cada 2 segundos), demostrando la eficiencia del modelo.

IV. EJERCICIO PRÁCTICO N.º 2: COMUNICACIÓN Y SINCRONIZACIÓN (PRODUCTOR-CONSUMIDOR)

IV-A. Objetivo y Arquitectura del Sistema

El segundo ejercicio aborda el problema clásico de sincronización y transferencia de información entre tareas independientes bajo el patrón **Productor-Consumidor**.

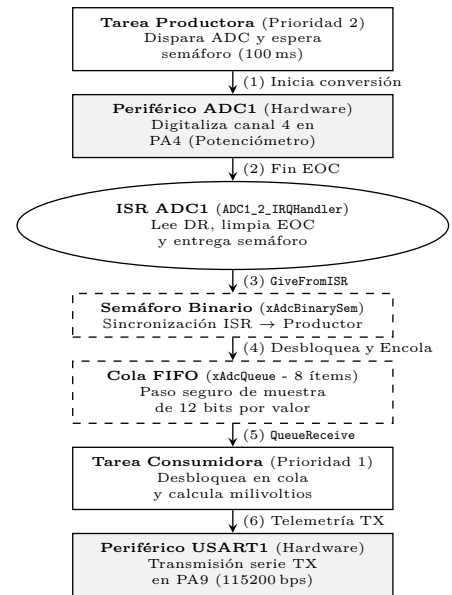


Figura 3. Flujo secuencial de sincronización, paso de datos e interacción con el hardware (Ejercicio 2).

El sistema se compone de los siguientes elementos:

1. **Entrada Analógica (ADC1 en PA4):** Se utiliza el canal 4 del ADC1 (ADC1_IN4) conectado al cursor central de un potenciometro alimentado entre 0 V y 3.3 V (según la disposición de hardware utilizada en el TP2). El reloj del ADC se ajusta a 12 MHz (prescaler

/6 desde APB2) con un tiempo de muestreo de 239.5 ciclos y la interrupción por fin de conversión (EOCIE) habilitada.

2. **Tarea Productora (vTaskProducerADC) – Prioridad 2:** Se ejecuta estrictamente cada 100 ms de manera periódica. Inicia la conversión del ADC por software y se bloquea a la espera de un **Semáforo Binario (xAdcBinarySem)**. Al ser notificada, toma el valor digitalizado (12 bits, 0 a 4095) y lo envía a una **Cola de Mensajes (xAdcQueue)**.
3. **Rutina de Servicio de Interrupción (ADC1_2_IRQHandler):** Al finalizar la conversión por hardware, la interrupción del ADC lee el registro de datos ADC1→DR (lo que limpia automáticamente la bandera EOC), entrega el semáforo binario mediante xSemaphoreGiveFromISR() y solicita el cambio de contexto con portYIELD_FROM_ISR().
4. **Tarea Consumidora (vTaskConsumerUART) – Prioridad 1:** Permanece bloqueada en la cola (xQueueReceive(..., portMAX_DELAY)). Al recibir una muestra, calcula la tensión equivalente en milivoltios y transmite la cadena formateada por la UART1 a 115200 bps.

IV-B. Implementación y Código Fuente

En el Listado 2 se detallan las funciones de sincronización e interrupción implementadas en act2/src/main.c.

Listing 2. Fragmento de Productor, Consumidor e ISR (act2/src/main.c)

```
static SemaphoreHandle_t xAdcBinarySem = NULL;
static QueueHandle_t xAdcQueue = NULL;
static volatile uint16_t g_adc_raw_value = 0;

void ADC1_2_IRQHandler(void) {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    if (ADC1→SR & ADC_SR_EOC) {
        g_adc_raw_value = (uint16_t)(ADC1→DR & 0xFFFF);
        if (xAdcBinarySem != NULL)
            xSemaphoreGiveFromISR(xAdcBinarySem, &
xHigherPriorityTaskWoken);
    }
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

static void vTaskProducerADC(void *pvParameters) {
    (void)pvParameters;
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(100);
    for (;;) {
        ADC1→CR2 |= ADC_CR2_ADON; // Inicia conversion
        if (xSemaphoreTake(xAdcBinarySem, pdMS_TO_TICKS(50)) ==
pdTRUE) {
            uint16_t muestra = g_adc_raw_value;
            xQueueSend(xAdcQueue, &muestra, 0);
            GPIOC→ODR ^= (1 << 13); // Toggle LED
        }
        vTaskDelayUntil(&xLastWakeTime, xPeriod);
    }
}

static void vTaskConsumerUART(void *pvParameters) {
    (void)pvParameters;
    uint16_t adc_val = 0;
    char msg_buf[64];
    for (;;) {
        if (xQueueReceive(xAdcQueue, &adc_val, portMAX_DELAY) ==
pdPASS) {
            uint32_t mV = ((uint32_t)adc_val * 3300UL) / 4095UL;
            snprintf(msg_buf, sizeof(msg_buf),
"ADC_PA4: %4u | Tension: %1u. %03luV\r\n",
adc_val, mV / 1000, mV % 1000);
        }
    }
}
```

```
        uart_send_str(msg_buf);
    }
}
```

IV-C. Análisis Conceptual: ¿Por qué se implementó así?

El diseño del Ejercicio 2 incorpora principios fundamentales de ingeniería de software embebido en tiempo real:

1. **Temporización periódica exacta con vTaskDelayUntil():** Para la adquisición de señales analógicas, la regularidad del período de muestreo ($T_s = 100$ ms) es crítica. Si se utilizara vTaskDelay(), el tiempo que demora la conversión del ADC y el encolado de datos se sumaría al retardo, produciendo corrimiento temporal acumulativo. vTaskDelayUntil() calcula el siguiente instante de despertar respecto a la marca temporal base xLastWakeTime, garantizando una frecuencia de muestreo de exactamente 10 Hz libre de deriva (*drift-free*).
2. **Patrón de Procesamiento Diferido (Deferred Interrupt Processing):** Mantener una ISR ejecutando rutinas prolongadas (como cálculos matemáticos o formateo de cadenas con snprintf y transmisión por UART) bloquearía a otras interrupciones del microcontrolador y arruinaría la capacidad de respuesta del sistema. Por ello, el procesamiento se divide en:
 - **Top-Half (La ISR):** Ejecuta en pocos ciclos de reloj: lee el registro DR, entrega el semáforo binario y concluye.
 - **Bottom-Half (La Tarea Consumidora):** Ejecuta a nivel de hilo (*Thread Mode*), encargándose de las operaciones de cómputo y transmisión UART de forma asíncrona sin afectar la latencia del hardware.
3. **Conmutación Inmediata con pxHigherPriorityTaskWoken:** Al entregar el semáforo dentro de la ISR mediante xSemaphoreGiveFromISR(), si la Tarea Productora (Prioridad 2) tiene mayor prioridad que la tarea que estaba corriendo antes de la interrupción, la variable xHigherPriorityTaskWoken pasa a valer pdTRUE. La macro portYIELD_FROM_ISR() fuerza la excepción PendSV, logrando que al salir de la interrupción el procesador ejecute de inmediato a la tarea productora sin tener que aguardar al próximo tick de reloj del kernel.
4. **Seguridad y Desacoplamiento mediante Colas (Pass by Value):** La cola almacena los datos realizando una copia directa byte a byte de la muestra en el Heap de FreeRTOS. Esto desacopla temporal y espacialmente al Productor del Consumidor: si el puerto UART se encuentra momentáneamente demorado, las muestras se acumulan ordenadamente en el buffer FIFO de la cola sin riesgo de condiciones de carrera ni corrupción de memoria.

V. RESULTADOS Y VALIDACIÓN EXPERIMENTAL

V-A. Compilación y Tamaño de Memoria

Ambos proyectos fueron compilados mediante la cadena de herramientas GNU GCC (`arm-none-eabi-gcc v14.2.0`) con optimizaciones activadas y eliminación de secciones no utilizadas (`-ffunction-sections`, `-fdata-sections`, `-Wl,-gc-sections`).

En la Tabla II se presentan los resultados obtenidos mediante la herramienta `arm-none-eabi-size` para los archivos ejecutables ELF finales.

Tabla II. Ocupación de memoria en Flash y SRAM

Proyecto	text (Flash)	data	bss (RAM)	Flash
Ejercicio 1	35856 B	1752 B	7288 B	37.6 KB / 64 KB
Ejercicio 2	39432 B	1752 B	7296 B	41.1 KB / 64 KB

La sección `.bss` refleja la asignación estática de los 5120 bytes destinados al Heap del sistema operativo más las variables de control de CMSIS y Newlib, manteniéndose holgadamente dentro de los 20 KB de memoria RAM física del microcontrolador.

V-B. Validación en Hardware

Tras grabar los microcontroladores mediante la interfaz OpenOCD y el programador ST-Link V2 (empleando el comando `set CPUPID 0x2ba01477`), se verificaron experimentalmente los siguientes comportamientos:

1. Ejercicio 1 (Blinky y Telemetría):

- Se confirmó visualmente el parpadeo simétrico del LED en PC13 a una frecuencia de 5 Hz (período de 200 ms).
- A través del terminal serie conectado a PA9 a 115200 bps, se verificó la recepción periódica cada 2 segundos del mensaje de telemetría:

```
=== TP4 Ejercicio 1: Tareas y Planificacion ===
Sistema Operativo Ejecutando idle:1420583
Sistema Operativo Ejecutando idle:2841170
Sistema Operativo Ejecutando idle:4261755
Sistema Operativo Ejecutando idle:5682341
```

El incremento uniforme y constante de $\approx 1,42 \times 10^6$ cuentas entre capturas valida que la CPU permanece en reposo ejecutando la tarea Idle entre eventos.

2. Ejercicio 2 (Productor-Consumidor):

- Al variar la posición del potenciómetro conectado a PA4 entre sus extremos, el terminal serie reflejó de manera continua y estable las lecturas digitalizadas (0 a 4095) y su cálculo correspondiente en voltios (0.000 V a 3.300 V):

```
=== TP4 Ejercicio 2: Productor-Consumidor ===
ADC PA4: 0 | Tension: 0.000 V
ADC PA4: 1241 | Tension: 1.000 V
ADC PA4: 2048 | Tension: 1.650 V
ADC PA4: 3103 | Tension: 2.500 V
ADC PA4: 4095 | Tension: 3.300 V
```

- El LED PC13 alternó su estado cada 100 ms confirmando la sincronización exacta del lazo de producción y la entrega exitosa del semáforo binario desde la interrupción del ADC.

VI. CONCLUSIÓN

A lo largo de este trabajo práctico se logró migrar exitosamente desde el paradigma clásico de programación secuencial hacia una arquitectura basada en un sistema operativo en tiempo real utilizando FreeRTOS sobre la plataforma STM32F103.

Se demostró cómo la infraestructura modular desarrollada en el Trabajo Práctico N.º 3 proporciona una base sólida para coordinar múltiples tareas con prioridades diferenciadas. Se verificó que el uso de primitivas de sincronización del sistema operativo (`vTaskDelay`, `vTaskDelayUntil`, semáforos binarios y colas) permite construir aplicaciones embebidas deterministas, libres de espera activa y protegidas contra condiciones de carrera, permitiendo al microcontrolador aprovechar al máximo los ciclos de CPU y responder de forma óptima a los eventos físicos del hardware.

REFERENCIAS

- [1] R. Barry, *Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide*, Real Time Engineers Ltd., 2016.
- [2] J. Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3rd ed., Newnes/Elsevier, 2014.
- [3] STMicroelectronics, *RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs*, DocID13902 Rev 21, 2021.
- [4] ARM Ltd., *CMSIS – Common Microcontroller Software Interface Standard*, ARM-Software GitHub Repository, 2024.